Food Store en la feria - TP8

Base de Datos II - UTN FRM - Noviembre 2026 Motores: PostgreSQL 18.6 (servidor de pruebas `foodstore_u8`) y SQLite 3.50.4 (`movil/foodstore_movil.db`).

1.a Que datos viajan a la tablet

Si viaja	No viaja
`producto_id`, `nombre`, `precio_centavos`, `stock_visto`, `disponible`, `eliminado`, `fecha_copia`	`cliente` (nombre, apellido, email, telefono), `usuario`/`contrasena`, historial de `pedido` y `detalle_pedido`

La tablet solo necesita el catalogo para consultar precios y armar borradores; no necesita identidades ni historial de ventas. **PostgreSQL es la fuente de verdad** de precio, stock y ventas: la tablet nunca confirma una venta ni modifica el servidor. El precio viaja en centavos para evitar errores de punto flotante y `disponible`/`eliminado` como `0/1`.

1.b Modelo local de la tablet

Tabla	Rol	Decisiones
`producto_cache`	Catalogo local	Precio en **centavos** (`INTEGER`, nunca `REAL`); `disponible`/`eliminado` como `0/1`; `fecha_copia` identifica la instantanea usada.
`pedido_borrador`	Cabecera del borrador	`id_local` UUID; `usuario` es el alias local `vendedor_feria` (sin PII); `forma_pago` cerrada por `CHECK`.
`item_borrador`	Lineas del borrador	PK `(id_local, producto_id)`; FK al borrador con `ON DELETE CASCADE` y a la cache con `ON DELETE RESTRICT`; `cantidad > 0`; precio visto congelado.
`cola_sync`	Solicitudes de sincronizacion	`solicitud_uuid` PK (idempotencia); estado cerrado por `CHECK`; `motivo`, `creado_en` y `evaluado_en`.
`snapshot_servidor`	Apoyo de importacion	Conserva los booleanos `t/f` del CSV de psql; no es parte del modelo operativo.

Las cuatro tablas operativas son `STRICT` y la base se abre con `PRAGMA foreign_keys = ON`. No existe tabla de clientes, usuarios ni ventas: la tablet no guarda PII ni historial.

1.c Transacciones y pruebas locales (C1 y C7)

Prueba	Resultado real
C1 catalogo consultado sin servidor	50.005 productos disponibles leidos de `producto_cache`, sin conexion a PostgreSQL.
Borrador confirmado con `COMMIT`	`pedido_borrador` conserva 1 fila para el UUID de prueba.
Operacion deshecha con `ROLLBACK`	0 filas: la transaccion no dejo rastro.
C7a cantidad 0	`CHECK constraint failed: cantidad > 0`; 0 filas despues del `ROLLBACK`.
C7b producto inexistente (999999)	`FOREIGN KEY constraint failed`; 0 filas despues del `ROLLBACK`.

Los dos errores de C7 no se corrigen: se demuestra que la base los rechaza. Evidencia: `movil/evidencias/casos_c1_c7.txt` (bloque `pruebas_sqlite.sql`).

2.a Instantanea y 2.b semaforo

`v_snapshot_movil` exporta 50.006 productos a `movil/servidor_snapshot.csv` (2,96 MB, `fecha_copia` 2026-11-12 12:08). El CSV se importa en `snapshot_servidor`; `semaforo.sql` normaliza los booleanos `t/f` de psql y actualiza `producto_cache` con `INSERT ... ON CONFLICT`. Luego evalua **solo** las solicitudes `PENDIENTE`: rojo `RECHAZADO`, amarillo `CONFLICTO`, verde `SIMULADO_APROBADO`. Repetir la corrida no reevalua lo ya evaluado.

2.c Casos C2 a C6

Caso	Comando / preparacion	Resultado real	Explicacion
C2	Borrador de producto 1 a 120000; `UPDATE producto SET precio = 1250.00 WHERE id = 1`; nueva instantanea; luego actualizar el borrador a 125000 y crear otra solicitud	`CONFLICTO: precio_visto_distinto=1`; la solicitud nueva: `SIMULADO_APROBADO`	El servidor subio el precio. El cliente acepta y se reenvia con el precio nuevo; la solicitud vieja queda en conflicto.
C3	Borrador de producto 2, cantidad 500, con stock visto 106	`RECHAZADO: stock_insuficiente=2`	La cantidad supera el stock de la instantanea; no se puede aprobar.
C4	Borrador de producto 3; `UPDATE producto SET activo = FALSE WHERE id = 3`; nueva instantanea	`RECHAZADO: eliminado=3`; cache: id 3, disponible 0, eliminado 1; catalogo disponible: 0 filas	La baja logica llega como lapida. La fila local se conserva para no romper el borrador, pero no aparece como disponible.
C5	Borrador de producto 4, cantidad 2, precio 85000, sin cambios	`SIMULADO_APROBADO`	Todos los datos coinciden con la instantanea. No es una venta.

Caso	Comando / preparacion	Resultado real	Explicacion
C6	Reenviar la solicitud de C5 con el mismo UUID y volver a correr el semaforo	`UNIQUE constraint failed: cola_sync.solicitud_uuid`; estado y `evaluado_en` sin cambios; 6 solicitudes	La PK impide duplicar. La segunda corrida no reevalua porque la solicitud ya no esta `PENDIENTE`. En un sistema real la proteccion es una clave de idempotencia en el servidor.

Verificacion en PostgreSQL

Metrica	Antes	Despues
Pedidos	227.009	227.009
Lineas de pedido	549.529	549.529
Productos	50.006	50.006
Stock total	4.989.376	4.989.376
Productos activos	50.006	50.005

Los pedidos y el stock no cambiaron. La unica diferencia es la baja logica del producto 3 que se preparo a proposito para C4. Evidencia: `movil/evidencias/servidor_antes.txt`, `servidor_despues.txt` y `casos_c1_c7.txt`.

3.a Mapa de la capa de datos

Motor	Que guarda	Quien manda
PostgreSQL	Catalogo, precios, stock, clientes, pedidos y lineas; es el sistema transaccional	**Fuente de verdad**. La aplicacion confirma la venta y descuenta stock aqui.
MongoDB (U7)	Documento de pedido con cliente esencial y lineas embebidas, para lectura y seguimiento	Modelo derivado. No decide precio ni stock; se replica desde PostgreSQL.
SQLite (tablet)	Cache de catalogo, borradores y cola de sincronizacion	**Nunca manda**. Consulta, propone y espera; `SIMULADO_APROBADO` no es una venta.

3.b Plan de mantenimiento

- **Respaldo PostgreSQL:** `pg_dump -Fc` diario y prueba de restauracion semanal en una base `*_restore_test`. Se validan conteos, se ejecuta `ANALYZE` y se reaplican roles/grants; el restore logico no transfiere estadisticas ni ACLs.

- **SQLite:** la base es descartable. Se copia el archivo y, si se pierde, se descarga una instantanea nueva; los borradores no confirmados se descartan.

- **MongoDB:** `mongodump`/`mongorestore` o reconstruccion desde PostgreSQL, porque es un modelo de lectura derivado.

- **Si se restaura un respaldo viejo de PostgreSQL:** las tablets deben dejar de sincronizar, descargar una instantanea nueva y volver a evaluar sus `PENDIENTE`. Un `SIMULADO_APROBADO` viejo no se puede presentar como venta; si el respaldo perdio ventas confirmadas, solo el servidor puede recuperarlas.

- **Revision de CONFLICTO/RECHAZADO:** al cierre de cada feria y en revision diaria por el responsable de datos (`admin_datos`), con prioridad para los rechazos por stock y las lapidas.

3.c Code review con IA (OpenCode, agente Plan)

Hallazgo	Decision	Motivo y comprobacion en la base
U6: el restore con `--no-owner --no-privileges` deja datos pero no ACLs	**Corregido**	En `foodstore_restore_test`, `has_table_privilege('app_web','pedido','SELECT') = f`. Se agrego al runbook reaplicar `sql/roles.sql` y grants despues del restore.
U4/TP6: `fecha = CURRENT_DATE` sobre `TIMESTAMPTZ` compara contra medianoche	**Corregido**	En `foodstore_u8`: `fecha = CURRENT_DATE` devuelve 0; el rango `fecha >= CURRENT_DATE AND fecha < CURRENT_DATE + 1` devuelve 1. Se usa rango.
U5: `ALTER DEFAULT PRIVILEGES` daba `SELECT` automatico a `rol_reportes`	**Corregido**	`pg_default_acl` tiene 0 filas y `rol_reportes` no puede leer `v_reporte_pedidos_cliente` (PII). Se dejo *default deny*.
U5/TP5: indice sobre `detalle_pedido(cantidad)`	**Descartado**	El plan de la agregacion sigue siendo `Seq Scan` antes y despues de crear el indice; solo agrega costo de escritura. Evidencia en `movil/evidencias/code_review_db.txt`.
U5: `fn_resetear_password` no restringe el rol destino	**Aceptado como riesgo documentado**	Un `rol_soporte` comprometido podria resetear un admin. Se mantiene por alcance y se propone token de un solo uso para produccion.

Cierre

El esquema, el semaforo y los casos C1 a C7 se ejecutaron sobre `foodstore_movil.db`; los conteos de PostgreSQL prueban que la tablet no altero pedidos ni stock. El estado final de `cola_sync` son 6 solicitudes: 2 `CONFLICTO`, 2 `RECHAZADO` y 2 `SIMULADO_APROBADO`. La solicitud de prueba de 1.c quedo en `CONFLICTO` al reevaluarse porque el precio del producto 1 ya habia subido en C2 (1200.00 visto contra 1250.00 de la instantanea): el semaforo evaluo una `PENDIENTE` real y no toco las ya cerradas. `PRAGMA integrity_check` devuelve `ok` y `PRAGMA foreign_key_check` no reporta filas.

La defensa puede mostrar C2 en vivo y el hallazgo descartado del indice. Requisitos y tareas: `docs/kiro/requirements.md`, `design.md` y `tasks.md`; bitacora y DUIA: `docs/bitacora_ia.md` y `docs/DUIA.md`.